



MÓDULO 4 · AULA 28

Aula 28

Testes com Python (pytest)

Testar as funções que limpam e transformam os dados.

Instrutor: Douglas Adriano Queiroz

O que vamos ver hoje



pytest

O framework de testes do Python



Anatomia do teste

test_ e assert



Testar transformações

Funções de limpeza



Fixtures

Dados de teste reutilizáveis



Casos de borda

Nulos, vazios, tipos



Boas práticas

Testes de dado que valem

pytest: testes simples em Python

O pytest é o framework mais usado para testar código Python — inclusive funções de dados.

Convenção: arquivos `test_*.py` e funções `def test_...()`.

Verificação: use `assert` — se for verdadeiro, passa; senão, falha.

Rodar: o comando `pytest` encontra e executa todos os testes.

Rápido: ideal para a base da pirâmide (muitos testes de unidade).



Analogia da cozinha

É provar cada ingrediente antes de juntar tudo: o sal está salgado? o limão, azedo? Testar cada função de dados isolada é provar um ingrediente de cada vez — se um está errado, você acha na hora.



Leve daqui: `pytest` + `assert`: a forma mais simples de provar que sua função de dados funciona.

Exemplo: testar uma função de limpeza

Uma função que remove linhas com preço inválido — e o teste que a prova.

```
lin peza.py + test_lin peza.py

# lin peza.py
import pandas as pd

def remove_precos_invalidos(df):
    # mantém só preço > 0 e não nulo
    return df[df['preco'].notna() & (df['preco'] > 0)]

# test_lin peza.py — prova a função
def test_remove_preco_invalido():
    df = pd.DataFrame({'preco': [10, 0, -5, None, 20]})
    out = remove_precos_invalidos(df)
    assert list(out['preco']) == [10, 20]
```



Leve daqui: A função de limpeza vira testável: entradas conhecidas, saída esperada exata.

Fixtures: dados de teste reutilizáveis

Uma fixture prepara um dado (ex.: um DataFrame de exemplo) para vários testes usarem.

@pytest.fixture: cria o dado uma vez e entrega pronto aos testes.

Evita repetição: o mesmo DataFrame de exemplo serve a muitos testes.

Deixa o teste limpo: cada teste foca no que verifica, não em montar dado.

Dados pequenos: use amostras mínimas que representem os casos.



Analogia da casa

É a mise en place da cozinha: deixar os ingredientes já lavados e cortados numa tigela, prontos para cada prato. A fixture é essa tigela de dados pronta.



Leve daqui: Fixture = dado de teste pronto e reutilizável — menos repetição, testes mais claros.

Exemplo: fixture + teste de transformação

```
test_transform acoes.py

import pandas as pd, pytest
from transform import normaliza_bairro

@pytest.fixture
def dados():
    return pd.DataFrame({'bairro': ['Centro ', 'trindade', 'CENTRO ']} )

def test_normaliza_bairro(dados):
    out = normaliza_bairro(dados) # tira espaços e padroniza
    assert list(out['bairro']) == ['Centro', 'Trindade', 'Centro']
```



Leve daqui: Padronizar texto (espaços, maiúsculas) é fonte clássica de bug em dados — teste sempre.

Boas práticas de teste unitário em dados



Teste as bordas

Nulos, vazios, zero, negativos, tipos errados — onde o dado 'quebra'.



Dados mínimos

Amostras pequenas e claras; não use a base inteira no teste.



Resultado exato

Compare com a saída esperada célula a célula, não 'mais ou menos'.



Determinístico

Fixe seeds (random_state) para o teste dar sempre o mesmo resultado.



Resumão da aula



pytest usa arquivos `test_*.py`, funções `test_` e `assert`.



Testes de unidade provam as funções de limpeza/transformação de dados.



Fixtures preparam dados de teste reutilizáveis (`@pytest.fixture`).



Sempre teste as bordas: nulos, vazios, zero, negativos, tipos.



Use dados mínimos e compare o resultado exato.



Fixe seeds para testes determinísticos.



Glossário da aula

pytest — Framework de testes do Python

assert — Verificação: verdadeiro passa, falso falha

Fixture — Dado/estado de teste preparado e reutilizável

DataFrame — Tabela de dados do pandas

Teste unitário — Testa uma função isolada

Caso de borda — Situação extrema (nulo, zero, vazio)

seed / random_state — Semente que torna o resultado reproduzível

Determinístico — Mesma entrada gera sempre a mesma saída

Transformação — Passo que altera/limpa os dados

Cobertura — Quanto do código os testes exercitam



Referências e para aprofundar

pytest Docs. docs.pytest.org — guia oficial de testes em Python.

Docs oficiais. pandas.pydata.org · scikit-learn.org · docs.pytest.org · greatexpectations.io · rapidminer.com

Trilha do curso: Data Science e Testes — Projeto Social Garapuvu 2026.